PocketGS: High-Fidelity On-Device Training for 3D Gaussian Splatting

Anonymous submission



Figure 1: PocketGS enables high-quality end-to-end 3DGS reconstruction entirely on a smartphone within a tight post-capture deployment envelope (approximately 4 minutes and less than 3 GB peak memory).

Abstract

Although 3D Gaussian Splatting (3DGS) supports real-time rendering, training relies on offline geometry, long-horizon densification, and memory-intensive differentiation. We cast post-capture static-scene training as resource-constrained finite-horizon optimization, where terminal quality depends jointly on the initial representation and reference-consistent gradient execution. To the best of our knowledge, PocketGS is the first publicly documented pipeline to complete geometry construction, Gaussian initialization, and 3DGS optimization on a commodity smartphone. Its $\mathcal{I} \circ \mathcal{G}$ warm start combines refined poses, a filtered dense prior, and surface-aligned anisotropic Gaussians. Its $\mathcal{T}$ operator replays compositing interactions and scatters gradients to canonical parameters, matching a numerical reference at the reported precision. Controlled 500-step experiments on LLFF, Mip-NeRF 360, and MobileScan show high-quality reconstruction at the device horizon. Across MobileScan, every scene remains below 3 GB peak memory and post-capture processing averages 255.2 s, enabling an on-device capture-to-render workflow. Separate 30k experiments characterize native-schedule behavior.

Introduction

3D Gaussian Splatting (3DGS) (Kerbl et al. 2023) combines high-fidelity scene modeling with real-time rendering for mixed reality, digital twins, and robotics. Its explicit, rasterization-friendly primitives replace implicit neural fields (Mildenhall et al. 2021) and motivate complete 3D reconstruction on commodity mobile devices. On-device processing benefits privacy-sensitive capture, poor-connectivity field work, and interactive AR requiring immediate feedback without uploading images or poses.

Despite efficient inference, 3DGS training assumes a workstation regime: offline SfM (Schönberger and Frahm 2016) supplies its input, long-horizon densification discovers scene structure, and differentiable rendering retains substantial intermediate state. Mobile time, memory, and thermal limits constrain both the number and cost of updates. Shortening the desktop schedule leaves weak geometry and initialization insufficiently corrected, while transferring its backward pass causes excessive memory traffic.

Recent work accelerates optimization (Mallick et al. 2024; Chen et al. 2025; Hanson et al. 2025), supports mobile ren-

dering and pruning (Lin, Feng, and Zhu 2025; Du et al. 2026; Sanim et al. 2025), or addresses large-scale deployment (Liu et al. 2025), but still relies on offline reconstruction or workstation-class training. To the best of our knowledge, PocketGS is the first publicly documented 3DGS pipeline to complete geometry construction, initialization, and optimization entirely on a commodity smartphone. We study post-capture reconstruction of static scenes rather than online SLAM or cross-device deployment.

This raises a central question: how should the initial representation and gradient execution be co-designed so that 3DGS reaches high-quality reconstruction within a device-determined number of updates and bounded memory? We view this as resource-constrained finite-horizon optimization. Under an update horizon K, the objective is reconstruction quality at the deployed iterate Θ_K, subject to end-to-end time and peak-memory constraints, rather than only long-horizon performance.

This perspective identifies representation formation and reference-consistent gradient execution as coupled optimization variables: scene structure must be introduced before the short horizon, while each update must remain affordable without approximating its gradient. For the former, mobile captures provide noisy poses and sparse geometry from on-device VIO (Qin, Li, and Shen 2018); standard 3DGS seeds isotropic Gaussians and relies on prolonged densification to recover structure (Kerbl et al. 2023). With small K, the starting state must instead provide reliable spatial support and encode local surface geometry before optimization. For the latter, differentiable splatting requires intermediate compositing states, although mobile GPUs expose only final blended outputs. Readback, full-cache maintenance, or recomputation can make each update prohibitively expensive.

PocketGS addresses these variables through three coupled operators. The geometry operator $\mathcal{G}$ refines poses and constructs a compact dense prior, reducing reliance on in-loop densification. The initialization operator $\mathcal{I}$ maps local surface statistics to surface-aligned anisotropic Gaussians, reducing scale and rotation correction during early updates. Together, $\mathcal{I} \circ \mathcal{G}$ provides the geometry-informed warm start Θ_0. The training operator $\mathcal{T}$ unrolls differentiable compositing, replays valid interactions, and maps their gradients to canonical parameters. It preserves reference gradients at the reported precision while reducing mobile memory traffic. This joint design targets Θ_K under one shared device budget rather than optimizing preprocessing, initialization, and training independently.

Our main contributions are:

- A geometry-informed warm-start design for finite-horizon 3DGS optimization: GPU-native pose/point refinement and single-reference MVS construct a compact dense prior, while surface-aligned covariance seeding moves structure formation before the iteration-limited optimization stage.
- A hardware-aligned differentiable splatting operator that replays valid-interaction state and resets only valid-entry counters, reducing mobile memory traffic while matching the numerical reference at the reported precision.
- To the best of our knowledge, PocketGS is the first publicly documented end-to-end 3DGS training pipeline to run fully on a commodity smartphone under strict runtime and memory constraints, enabling on-device capture-to-render workflows.

Experiments on LLFF, Mip-NeRF 360, and MobileScan show that PocketGS produces high-quality reconstructions under its intended 500-step training budget. Every evaluated MobileScan scene remains below 3 GB peak memory, a representative post-capture run takes approximately four minutes, and $\mathcal{T}$ matches the numerical reference at the reported precision. Native-schedule workstation experiments separately characterize long-horizon behavior.

Related Work

Novel View Synthesis and Gaussian Splatting

Novel view synthesis (Tewari et al. 2022; Gao et al. 2022) progressed from neural radiance fields (Mildenhall et al. 2021; Barron et al. 2021, 2022) and explicit grids or hashes (Fridovich-Keil et al. 2022; Sun, Sun, and Chen 2022; Müller et al. 2022; Barron et al. 2023; Guo, Wang, and Chen 2025) to real-time 3D Gaussian Splatting (Kerbl et al. 2023; Fang and Wang 2024a,b). Subsequent work improves SLAM integration, geometry, and optimization (Peng et al. 2024; Yan et al. 2024; Wenzhi et al. 2023; Huang et al. 2024; Chen et al. 2024; Höllein et al. 2025; Chen et al. 2025; Fang and Wang 2025), but generally retains offline SfM (Triggs et al. 1999; Schönberger and Frahm 2016) and assumes poses and an initial point set. PocketGS instead places pose refinement and dense-prior construction inside on-device training.

Efficient and Mobile Gaussian Splatting

Efficient 3DGS reduces training or memory through pruning and adaptive optimization (Mallick et al. 2024; Lan et al. 2025). EDGS and Turbo-GS study densification, initialization, and resolution trade-offs (Kotovenko, Grebenkova, and Ommer 2025; Lu et al. 2024); RAIN-GS handles inaccurate clouds (Jung et al. 2024), and Mini-Splatting2 accelerates workstation training (Fang and Wang 2024b). These accelerators retain workstation execution or offline inputs. Mobile-GS targets mobile rendering kernels (Du et al. 2026), with extensions to larger deployments and mobile perception (Liu et al. 2025; Liu, Shi, and Yang 2025). VIO/MVS scanning stacks (Picard et al. 2023; Qin, Li, and Shen 2018; Yao et al. 2018; Kim et al. 2022; Feigl et al. 2020; Mur-Artal and Tardós 2017) and mobile neural-rendering systems (Chen et al. 2023; Rojas et al. 2023) provide geometry or inference, but do not jointly connect mobile geometry recovery to trainable 3DGS primitives and device-side backpropagation. PocketGS couples prior construction, Gaussian initialization, and backpropagation within one phone resource envelope.

PocketGS: On-Device 3DGS Training

PocketGS performs end-to-end on-device 3DGS training for static scene reconstruction from handheld captures. Given an RGB sequence $\{I_t\}_{t=1}^n$, coarse poses $\{\hat{T}_t\}_{t=1}^n$ from a mobile tracker (*e.g.*, ARKit, ARCore), and a device-determined

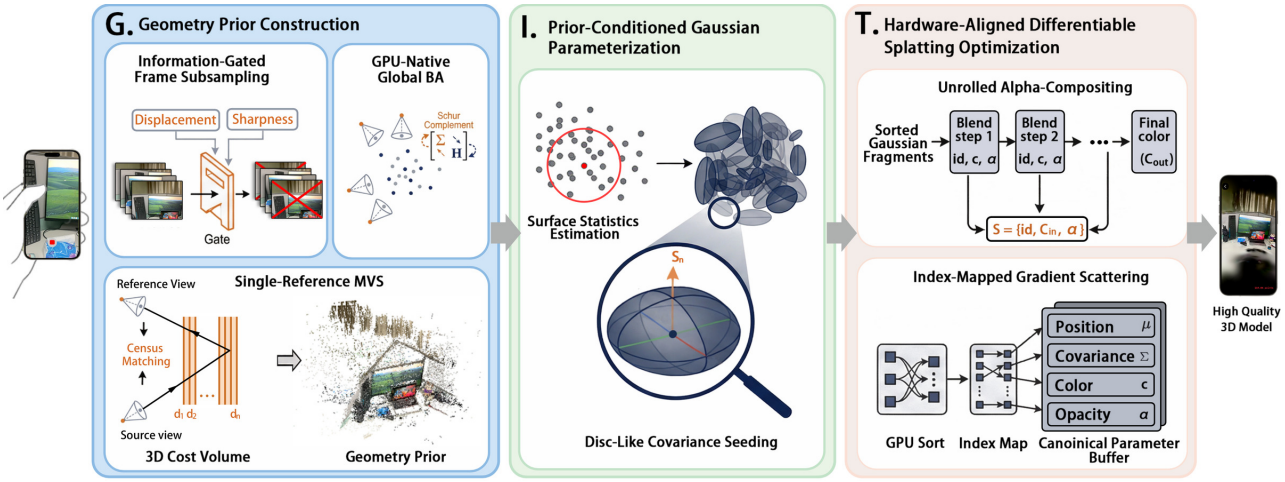


Figure 2: **Overview of the PocketGS framework.** PocketGS tackles on-device 3DGS training through three coupled operators: ($\mathcal{G}$) Geometry Prior Construction combines information-gated frame selection, GPU-native Schur-complement BA, and single-reference MVS to construct a dense geometric prior. ($\mathcal{I}$) Prior-Conditioned Parameterization seeds anisotropic Gaussians by estimating local surface statistics (normals $\mathbf{n}_i$ and scales) to front-load structure discovery via disc-like covariance seeding. ($\mathcal{T}$) Hardware-Aligned Splatting implements a mobile-native differentiable renderer using unrolled alpha-compositing ($S = id, C_{in}, \alpha$) and index-mapped gradient scattering for reference-consistent gradient execution at the reported precision within the mobile memory bounds of the canonical parameter buffer (μ, Σ, c, α) during end-to-end training.

update horizon K, it outputs a Gaussian set Θ_K for novel view synthesis under end-to-end time and peak-memory constraints on the target device.

PocketGS comprises three coupled operators. Their finite-horizon roles are

$$\begin{aligned}(\{T_t\}, P) &= \mathcal{G}(\{I_t, \hat{T}_t\}), & \Theta_0 &= \mathcal{I}(P), \\ g_k &= \mathcal{T}(\Theta_k, \{I_t, T_t\}), & \Theta_{k+1} &= \mathrm{Adam}(\Theta_k, g_k),\end{aligned} \tag{1}$$

for $k = 0, \ldots, K-1$. Thus $\mathcal{I} \circ \mathcal{G}$ sets the geometry-informed warm start Θ_0, while $\mathcal{T}$ supplies the reference-consistent gradient for each update; both are co-designed under the shared mobile device budget.

Geometry Prior Construction under Resource Budgets

Standard 3DGS relies on prolonged optimization and training-time densification to recover structure from noisy, sparse inputs, causing unstable geometry and excessive mobile cost. Before $k = 0$, $\mathcal{G}$ instead refines $\{T_t\}$ and constructs a filtered dense prior P that supplies spatial support to $\mathcal{I}$, reducing reliance on in-loop densification and stabilizing early finite-horizon updates on mobile hardware.

Information-Gated Frame Subsampling Handheld captures suffer from motion blur, defocus, and temporal redundancy. We therefore retain only sharp, geometrically useful frames via a two-stage information gate.

Displacement Gate. Let $\mathbf{t}_{curr}$ and $\mathbf{t}_{last}$ be the camera centers of the current and last selected keyframes. A frame is kept only if

$$d = \|\mathbf{t}_{curr} - \mathbf{t}_{last}\|_2 \;\geq\; \tau_d,$$

with $\tau_d = 0.05\,\mathrm{m}$, which suppresses redundant viewpoints while preserving parallax for triangulation.

Sharpness Heuristic. For frames admitted by the displacement gate, we reject blurred ones via gradient energy on a sparse luma grid Ω with stride Δ:

$$\begin{aligned}S = \frac{1}{|\Omega|} \sum_{(x,y)\in\Omega} \big(&|I(x+\Delta, y) - I(x, y)| \\ &+ |I(x, y+\Delta) - I(x, y)|\big),\end{aligned} \tag{2}$$

favoring frames with stronger high-frequency content.

Candidate Windowing. Within each 0.25-s window, we retain the sharpest of up to 8 eligible frames.

GPU-Native Global Bundle Adjustment Mobile tracking poses are typically too noisy for 3DGS. We therefore run a GPU-native global BA initialized from $\{\hat{T}_i\}$ to jointly refine poses and sparse 3D points using the robust reprojection objective below:

$$\min_{\{T_i\},\{P_j\}} \sum_{(i,j)\in\mathcal{O}} \rho\Big(\big\|\pi(T_i, P_j) - \mathbf{p}_{ij}\big\|^2_{W_{ij}}\Big), \tag{3}$$

where π is the camera projection, $T_i \in SE(3)$ and $P_j \in \mathbb{R}^3$ the pose and point parameters, $\mathbf{p}_{ij} \in \mathbb{R}^2$ the measured image location, $W_{ij} = \Sigma_{ij}^{-1}$ the information weighting, $\mathcal{O}$ the valid-observation set, and $\rho(\cdot)$ the Huber loss.

Scale-Aware Gauge Fixing. The SfM objective is unchanged by a global similarity transform, giving a 7-DoF ambiguity (three rotation, three translation, and one scale). We remove it by fixing the first keyframe's 6-DoF pose and the first-to-second baseline length to the metric value supplied by the mobile tracker, yielding a stable metric frame.

GPU-Native Schur Complement.

The BA bottleneck is the normal equation $\mathbf{H}\Delta = \mathbf{b}$ with $\mathbf{H} = \mathbf{J}^\mathsf{T}\mathbf{J}$. Partitioning Δ into pose (t) and point (p) blocks yields the reduced camera system

$$(\mathbf{H}_{tt} - \mathbf{H}_{tp}\mathbf{H}_{pp}^{-1}\mathbf{H}_{pt})\Delta_t = \mathbf{b}_t - \mathbf{H}_{tp}\mathbf{H}_{pp}^{-1}\mathbf{b}_p. \quad (4)$$

Since $\mathbf{H}_{pp}$ is block-diagonal with independent 3×3 blocks, $\mathbf{H}_{pp}^{-1}$ is computed fully in parallel on the GPU, and Δ_p is recovered by back-substitution from Δ_t.

Iterative Geometric Refinement. BA is embedded in a refinement loop: after each global update we re-triangulate features with the updated poses, drop points with high reprojection error or insufficient triangulation angle, and discard behind-camera observations, purifying the sparse scaffold for MVS and subsequent Gaussian initialization.

Single-Reference Cost-Volume MVS Mobile tracking clouds are too sparse to initialize 3DGS reliably. We address this with a lightweight MVS stage that recovers a denser geometric prior under mobile budgets.

Probabilistic Depth Range Estimation. Plane-sweep efficiency hinges on the depth range. For each target frame we project sparse BA points, take the 5% and 95% depth quantiles $q_{.05}, q_{.95}$, and set the search range to $[q_{.05}\cdot 0.5,\ q_{.95}\cdot 1.5]$, intentionally widened to tolerate BA noise while focusing on the most probable scene volume under the fixed mobile compute budget available on-device.

Memory-Efficient Cost Volume. To minimize memory, each target view uses a single reference, chosen to maximize

$$S_{ref} = \exp\left(-\frac{(b - b_{target})^2}{2\sigma_b^2}\right) \cdot \min\left(\frac{\alpha}{\alpha_{min}}, 1\right), \quad (5)$$

where b is the candidate baseline, b_{target} the preferred baseline, and α the viewing angle, favoring informative parallax without overly wide baselines that hurt photometric consistency across the selected handheld views.

Matching cost uses the Census Transform (Fife and Archibald 2012) for illumination robustness and plane-sweep depth (Collins 1996) with SGM aggregation (Hirschmuller 2005). Depth maps are filtered at confidence 0.4 (a robust density–outlier trade-off) and fused into P. The resulting dense geometry provides the spatial support from which $\mathcal{I}$ constructs the geometry-informed warm start Θ_0.

Prior-Conditioned Gaussian Parameterization

Standard 3DGS initializes isotropic Gaussians from inter-point distances, leaving local surface scale and rotation to be recovered during optimization. Under a short K, these corrections consume available updates. $\mathcal{I}$ instead maps P to a surface-aligned anisotropic Θ_0, providing spatial support, scale, and rotation before optimization and supporting rapid early convergence within the fixed horizon.

Local Surface Statistics Estimation For each $\mathbf{p}_i$ in the dense cloud we estimate surface statistics from two KNN neighborhoods of deliberately different sizes: $K_n = 16$ for the normal (a larger neighborhood suppresses sampling noise) and $K_s = 3$ for the tangential scale which must track local density to avoid over-smoothing thin structures. Using $\{\mathbf{p}_{i,k}\}_{k=1}^{K_n}$ with centroid $\bar{\mathbf{p}}_i = \frac{1}{K_n}\sum_{k=1}^{K_n}\mathbf{p}_{i,k}$, we compute the local covariance matrix as

$$\mathbf{C}_i = \frac{1}{K_n}\sum_{k=1}^{K_n}(\mathbf{p}_{i,k} - \bar{\mathbf{p}}_i)(\mathbf{p}_{i,k} - \bar{\mathbf{p}}_i)^\mathsf{T}, \quad (6)$$

whose smallest-eigenvalue eigenvector gives the local normal $\mathbf{n}_i$. This fixed-K scheme is embarrassingly parallel and memory-bounded, fitting mobile GPUs naturally.

Disc-Like Covariance Seeding Each Gaussian is seeded as a thin disc-like ellipsoid tangent to the local surface, injecting geometry-aware anisotropy directly into the parameterization. The tangential scale is the average distance to the K_s nearest neighbors within the dense prior,

$$s_t = \frac{1}{K_s}\sum_{k=1}^{K_s}\|\mathbf{p}_i - \mathbf{p}_{i,k}\|, \quad (7)$$

and the normal-direction scale is $s_n = r_{normal}\, s_t$, giving a thin surface-aligned primitive with enough thickness for stable training. Scales are optimized in log-space for positivity; the initial rotation $\mathbf{q}_i$ aligns the Gaussian's z-axis with $\mathbf{n}_i$, and opacity is initialized to the logit of 0.1. Estimated normals serve only for initialization, never as training constraints; they define the surface-aligned rotation in the geometry-informed $\Theta_0 = \mathcal{I}(P)$, which matters most under strict iteration budgets.

Hardware-Aligned Differentiable Splatting Optimization

Each update requires a differentiable-rendering gradient, yet mobile training is bandwidth-bound: fixed-function blending exposes only the final color, whereas backpropagation needs intermediate compositing states. We implement $g_k = \mathcal{T}(\Theta_k, \{I_t, T_t\})$ by unrolling compositing and caching a compact GPU replay trace without framebuffer readback or backward-time sequence reconstruction. This changes retained state and execution, not the objective or the alpha-compositing derivatives of the standard renderer.

Unrolled Alpha-Compositing with Forward Replay Cache We unroll depth-ordered source-over compositing into an explicit graph and store a compact per-pixel replay trace containing the fragment identity and required state.

Explicit Computation Graph. For depth-sorted visible Gaussians, we explicitly compose

$$C_{out} = C_{in}(1 - \alpha) + \alpha\, c, \quad (8)$$

where C_{in} is the incoming accumulator and (α, c) are the fragment's opacity and color. The same unrolled chain yields the following closed-form gradients:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial C_{in}} &= \frac{\partial \mathcal{L}}{\partial C_{out}}(1 - \alpha), \quad \frac{\partial \mathcal{L}}{\partial c} = \alpha\, \frac{\partial \mathcal{L}}{\partial C_{out}}, \\ \frac{\partial \mathcal{L}}{\partial \alpha} &= \left\langle \frac{\partial \mathcal{L}}{\partial C_{out}},\ c - C_{in} \right\rangle, \end{aligned} \quad (9)$$

where $\langle\cdot,\cdot\rangle$ is the per-channel inner product. These gradients are then scattered to the canonical Gaussian parameter buffer through the index-mapped scattering operation described next, providing mobile gradient execution under strict device constraints.

Method	LLFF					Mip-NeRF 360					MobileScan				
	PSNR↑	SSIM↑	LPIPS↓	Time(s)	Count	PSNR↑	SSIM↑	LPIPS↓	Time(s)	Count	PSNR↑	SSIM↑	LPIPS↓	Time(s)	Count
3DGS-SFM-WK	23.28	0.789	0.226	116.3	18k	21.05	0.649	0.377	398.7	46k	22.64	0.781	0.319	123.8	23k
3DGS-MVS-WK	21.04	0.730	0.262	320.3	41k	18.26	0.586	0.429	975.6	76k	22.23	**0.834**	0.226	547.5	165k
Mobile-GS	23.26	0.790	0.227	133.1	18k	21.06	0.650	0.377	405.2	46k	22.61	0.782	0.321	145.1	23k
DashGaussian	23.30	0.790	0.224	115.9	18k	21.54	0.672	0.351	399.9	46k	22.66	0.783	0.317	122.5	23k
Taming-3DGS	23.27	0.788	0.228	117.6	18k	21.09	0.651	0.375	402.6	46k	22.62	0.780	0.322	125.1	23k
FastGS	23.31	0.788	0.227	116.3	18k	21.12	0.652	0.373	396.4	46k	22.63	0.780	0.320	123.9	23k
Speedy-Splat	23.29	0.789	0.225	133.1	18k	21.07	0.650	0.376	425.8	46k	22.65	0.781	0.319	144.5	23k
PocketGS (Ours)	**23.54**	**0.791**	**0.222**	105.4	33k	**22.25**	**0.702**	**0.302**	145.5	77k	**23.67**	0.791	**0.225**	255.2	168k

Table 1: **Reconstruction at a controlled 500-step horizon.** Dataset-level averages on LLFF, Mip-NeRF 360, and MobileScan with densification and pruning disabled for every variant. The experiment measures terminal quality at $K = 500$ from each method's preprocessing and initialization pipeline, before adaptive density control. Completed native-schedule behavior is evaluated separately. Time reports per-system post-capture processing latency on the stated hardware. Higher is better for PSNR/SSIM, and lower is better for LPIPS. Red, orange, and yellow denote the best, second-best, and third-best results.

Bandwidth-Efficient Forward Replay Cache. To avoid reconstructing per-pixel splat sequences during backprop, we record only the replay-essential quantities

$$S = \{\mathrm{id},\, C_{in},\, \alpha\}, \tag{10}$$

where id is the sorted splat identifier used to scatter gradients. A per-pixel counter $\mathrm{count}(u)$ tracks valid entries, and $K_{\max}$ is the per-pixel replay capacity. Each iteration resets only the $O(WH)$ counters rather than the full $O(WHK_{\max})$ cache, and backprop traverses only $\ell \in [0, \mathrm{count}(u)-1]$. Since stale entries outside this range are never read, counter reset leaves the explicit compositing gradient unchanged while avoiding a bandwidth-heavy cache clear; reference agreement is reported in the operator evaluation.

Index-Mapped Gradient Scattering Depth-sorted rendering permutes Gaussians, and physically reordering parameter buffers would incur heavy data movement and risk misaligning optimizer states. We therefore decouple the *sorted view* (rendering/backprop) from the *canonical layout* (parameters and optimizer moments). A GPU sort yields a permutation π mapping sorted index i to canonical index $\pi(i)$; kernels operate on the sorted view while gradients are scattered back as

$$\nabla\theta_{\pi(i)} \mathrel{+}= g_i, \tag{11}$$

with g_i the gradient of the i-th sorted element, keeping Adam states aligned with their parameters without CPU intervention or redundant memory copies.

Together, $\mathcal{G}$ and $\mathcal{I}$ form Θ_0 before the finite horizon, while $\mathcal{T}$ executes each reference-consistent update within mobile memory. PocketGS thus co-designs Θ_0 and the K reference-consistent updates yielding the deployed iterate Θ_K.

Experiments

Platform and budget. PocketGS is implemented in Swift on Apple Metal and runs entirely on an iPhone 15 (Apple A16, 6 GB unified memory). We evaluate two complementary regimes. The controlled 500-step study matches image sets, held-out splits, resolution, metrics, and optimization steps while retaining each system's stated preprocessing and initialization; densification and pruning are disabled to evaluate terminal quality at the finite horizon $K = 500$. For PocketGS, this lies within its intended five-minute post-capture compute budget, measured from loaded inputs to the trained model. The 30k workstation study retains every optimizer's native density-control and scheduling mechanisms to characterize long-horizon behavior. The former reports geometry, initialization, and optimization time; the latter reports training-script wall-clock time and excludes geometry and initialization-data generation. Full protocol details are provided in the supplement. Unless noted otherwise, quantitative results report means over five runs.

Baselines. Geometry baselines feed COLMAP SfM (*3DGS-SFM-WK*) or dense MVS (*3DGS-MVS-WK*) into vanilla 3DGS (Schönberger and Frahm 2016). Training baselines use the same SfM prior with Mobile-GS (Du et al. 2026), DashGaussian (Chen et al. 2025), Taming-3DGS (Mallick et al. 2024), FastGS (Ren et al. 2025), and Speedy-Splat (Hanson et al. 2025). COLMAP uses two RTX 3090 GPUs; PocketGS instead co-designs its prior and initialization. Runtime reports per-system latency on the stated hardware; we do not use cross-platform time for speedup rankings.

Datasets and metrics. We evaluate on Mip-NeRF 360 (Barron et al. 2022), LLFF (Mildenhall et al. 2019), and our phone-captured MobileScan. We uniformly downsample LLFF by $32\times$ and Mip-NeRF 360 by $16\times$; the latter uses 100 frames per scene. MobileScan contains 16 varied scenes, each with 50 smartphone keyframes and ARKit intrinsics, poses, and sparse structure; every eighth frame is held out. Image sets, resolutions, held-out splits, metrics, and step counts are matched across methods; geometry and pose sources follow each stated preprocessing pipeline, and public splits follow 3DGS (Kerbl et al. 2023). Public benchmarks provide the primary cross-method evidence; MobileScan tests the complete capture pipeline, device resources, and ablations under realistic pose noise. For the controlled results, we report PSNR, SSIM, LPIPS, Gaussian count, and post-capture processing time $T_{\text{geom}} + T_{\text{init}} + T_{\text{train}}$. Per-scene

Method	Step	PSNR↑	SSIM↑	LPIPS↓
3DGS-SFM-WK	30k	25.84	0.847	0.122
	15k	25.71	0.845	0.127
	7k	25.38	0.832	0.146
	3k	24.44	0.800	0.181
	1k	22.83	0.719	0.287
PocketGS	**500**	**22.25**	**0.702**	**0.302**

Table 2: **Early convergence on Mip-NeRF 360.** PocketGS's controlled 500-step endpoint versus the 1k–30k native-schedule checkpoints of 3DGS-SFM-WK; complete native trajectories are reported in the supplement.

results and full 500–30k trajectories are provided in the supplement.

Main Results

Table 1 and Fig. 3 summarize reconstruction at the controlled 500-step horizon. Because adaptive density control is disabled, the final Gaussian counts are outcomes of the respective priors rather than matched inputs. At this shared horizon, PocketGS attains the best reported PSNR and LPIPS across all three datasets. Training accelerators remain closely clustered at this early stage; their long-horizon trajectories are reported in the supplement. Corresponding qualitative comparisons on LLFF and Mip-NeRF 360 are provided in Supplementary Figs. 4 and 6, respectively.

LLFF. PocketGS leads all three quality metrics (23.54 dB/0.791/0.222) in 105.4 s. Although 3DGS-MVS-WK uses more Gaussians (41k versus 33k), it reaches only 21.04 dB, showing that a larger generic cloud does not ensure better 500-step reconstruction.

Mip-NeRF 360. PocketGS reaches 22.25 dB/0.702/0.302, improving over the strongest 500-step baseline, DashGaussian, by 0.71 dB/0.030/0.049. The 18.26 dB 3DGS-MVS-WK result again shows weak early support from generic dense initialization on unbounded scenes.

MobileScan. PocketGS gives the highest PSNR (23.67 dB) and lowest LPIPS (0.225), exceeding DashGaussian by 1.01 dB. Its 168k Gaussians nearly match 3DGS-MVS-WK's 165k, separating quality from model count. Figure 3 shows better preservation of textures and thin structures across difficult regions.

Across all three evaluated datasets, PocketGS produces high-quality 3DGS models at its intended 500-step on-device budget, demonstrating rapid early convergence in the target mobile reconstruction regime on the tested smartphone.

Convergence beyond 500 steps. Table 2 compares PocketGS's controlled 500-step endpoint with native-schedule 3DGS-SFM-WK at 1k–30k on Mip-NeRF 360.

With the scene set, resolution, split, and metrics held fixed, PocketGS's controlled 500-step endpoint approaches native-schedule 3DGS-SFM-WK at 1k (22.25/0.702/0.302 versus 22.83/0.719/0.287 in PSNR/SSIM/LPIPS), indicating faster early convergence from the complete PocketGS preprocessing and initialization pipeline. Because geometry processing, pose refinement, and Gaussian initialization are co-designed, this result measures their pipeline-level early-optimization benefit.

Method	On-device Train	FPS↑
MobileNeRF (Chen et al. 2023)	×	31.40
MetaSapiens (Lin, Feng, and Zhu 2025)	×	49.24
Mobile-GS (Du et al. 2026)	×	62.56
PocketGS (Ours)	✓	52.32

Table 3: **Smartphone rendering.** Baselines are rendering-only references under their reported setups; PocketGS is measured on the tested smartphone and trains fully on-device.

In the complementary 30k native-schedule study on Mip-NeRF 360 with ordinary COLMAP initialization, Taming-3DGS gives the highest PSNR (26.71 dB), DashGaussian the best SSIM/LPIPS (0.868/0.100), and FastGS the lowest training-script wall-clock time (240.3 s); no method dominates all completed-training criteria.

On-device Rendering and Memory

On-device rendering. Table 3 summarizes reported smartphone rendering throughput. PocketGS trains on-device and renders at 52.32 FPS; baselines are rendering-only references with offline-prepared models under their respective reported setups. To the best of our knowledge, PocketGS is the first publicly documented 3DGS pipeline combining complete on-device training with real-time rendering. Supplementary Fig. 5 shows MobileScan, LLFF, and Mip-NeRF 360 examples produced by PocketGS on the tested smartphone.

Memory footprint. Geometry construction averages 1.48 GB peak memory and training averages 2.22 GB; every MobileScan scene stays below 3 GB. On the 50-image Desk scene, a complete run takes 4.3 minutes, consumes about 2% battery, and reaches 42°C without throttling. Per-scene and stress-test results are in the supplement.

Ablation Study

Table 4 isolates the warm-start components on MobileScan. The w/o global BA and w/o MVS variants ablate two stages of $\mathcal{G}$, while w/o initialization directly tests $\mathcal{I}$. Without $\mathcal{I}$, the optimizer must correct scale and rotation within the limited budget, reducing quality and increasing runtime. Removing global BA mainly reduces structural coherence (SSIM $0.791 \rightarrow 0.760$), whereas removing MVS causes the largest quality loss (PSNR $23.67 \rightarrow 21.15$). Its shorter runtime reflects only the smaller primitive set.

Hardware-aligned operator $\mathcal{T}$. Relative to full-cache clear, replay-cache/counter-reset reduces backward time from 21.8 to 15.2 ms, isolated $\mathcal{T}$-path latency from 43.7 to 35.9 ms, and peak memory from 2.41 to 2.21 GB with zero measured gradient error at the reported precision. These operator-level latencies exclude the surrounding optimizer

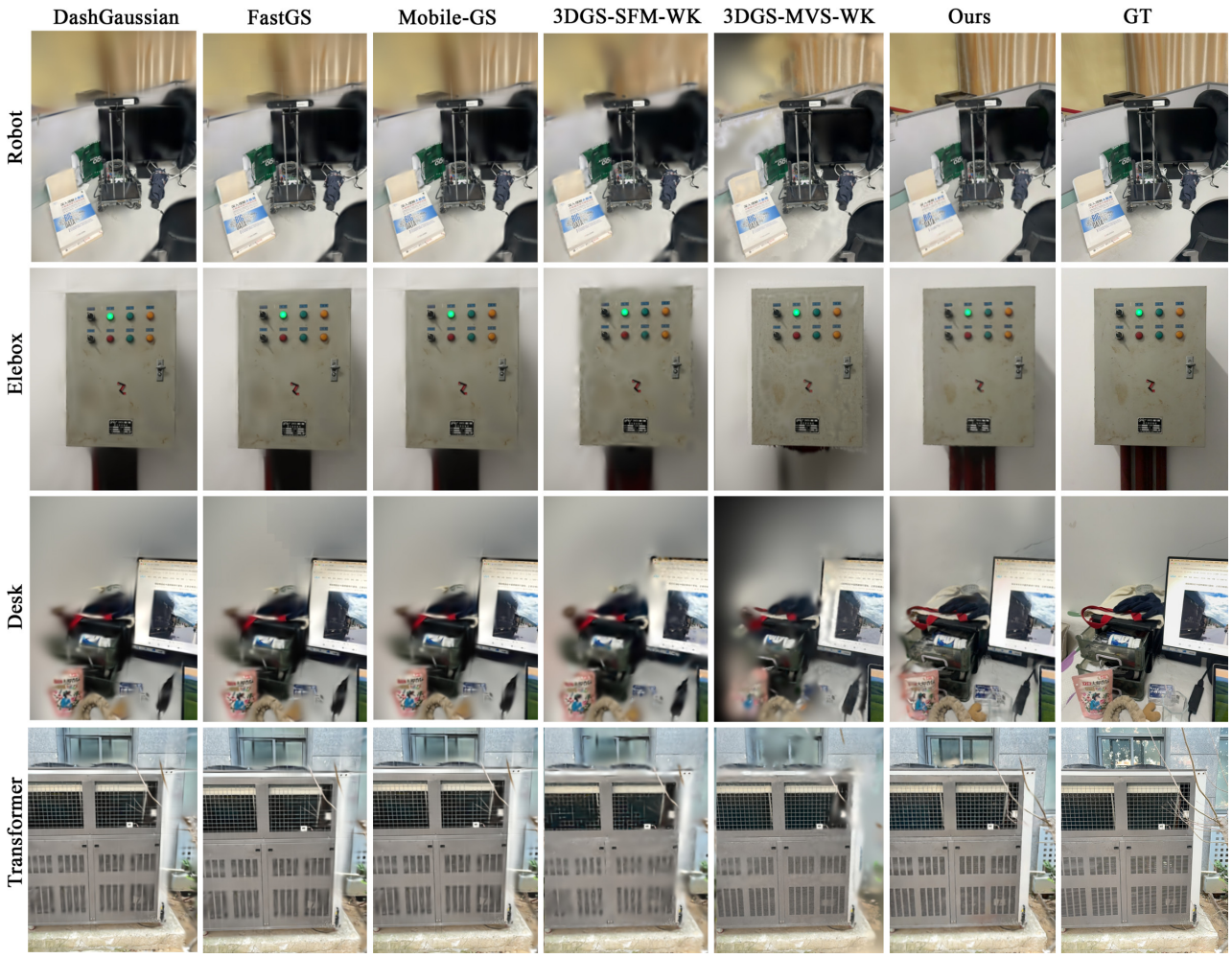


Figure 3: MobileScan reconstruction after 500 optimization steps. PocketGS better preserves textures and thin structures; generic MVS remains biased toward textured regions, leaving low-texture and specular areas under-sampled at initialization.

Variant	PSNR↑	SSIM↑	LPIPS↓	Time↓ (s)
PocketGS (Full)	**23.67**	**0.791**	**0.225**	255.2
w/o Global BA	23.54	0.760	0.227	251.8
w/o MVS	21.15	0.667	0.400	**127.1**
w/o Init. $\mathcal{I}$	22.49	0.778	0.249	317.0

Table 4: **MobileScan ablation.** Averages over 16 scenes under the matched 500-step protocol; the w/o MVS runtime reflects its smaller initial primitive count.

update and training-loop work. The method resets only valid-entry counters and replays the compact forward trace, avoiding a bandwidth-heavy global cache clear. CPU readback instead reaches 3.84 GB, while naive recomputation (Kerbl et al. 2023) takes 32.4 ms backward.

Geometry prior vs. COLMAP On MobileScan, COLMAP SfM takes 98.3 s for about 23k sparse points; adding dense MVS raises pipeline time to 619.8 s for 165k points. PocketGS records 31.57 s for a comparable 168k-point smartphone prior. These respective-platform latencies are not used for speedup rankings. GPU-native BA, single-reference MVS, and prior-conditioned parameterization are co-optimized with training.

Per-scene, operator, runtime, sensitivity, and memory analyses are reported in the supplement.

Conclusion

We present **PocketGS**, an end-to-end 3DGS training framework for static-scene reconstruction on a smartphone. Its geometry-informed warm start $\mathcal{I} \circ \mathcal{G}$ seeds surface-aligned Gaussians from a filtered dense prior, while $\mathcal{T}$ provides bandwidth-efficient, reference-consistent mobile gradient execution. Public benchmarks support the controlled 500-step comparison; MobileScan evaluates the phone-capture pipeline. On the tested smartphone, PocketGS averages four minutes post-capture, remains below 3 GB peak memory, and supports real-time rendering. Limitations include one mobile SoC, static scenes, and mobile-resolution inputs.

References

Barron, J. T.; Mildenhall, B.; Tancik, M.; Hedman, P.; Martin-Brualla, R.; and Srinivasan, P. P. 2021. Mip-NeRF: A Multiscale Representation for Anti-Aliasing Neural Radiance Fields. In *ICCV*.

Barron, J. T.; Mildenhall, B.; Tancik, M.; Srinivasan, P. P.; Han, X.; and Martin-Brualla, R. 2022. MipNeRF 360: Unbounded anti-aliased neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Chen, D.; Li, H.; Ye, W.; Wang, Y.; Xie, W.; Zhai, S.; Wang, N.; Liu, H.; Bao, H.; and Zhang, G. 2024. PGSR: Planar-based Gaussian Splatting for Efficient and High-Fidelity Surface Reconstruction. *IEEE TVCG*.

Chen, Y.; Jiang, J.; Jiang, K.; Tang, X.; Li, Z.; Liu, X.; and Nie, Y. 2025. DashGaussian: Optimizing 3D Gaussian Splatting in 200 Seconds. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 11146–11155.

Chen, Z.; Funkhouser, T.; Hedman, P.; and Tagliasacchi, A. 2023. Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16569–16578.

Collins, R. T. 1996. A space-sweep approach to true multi-image matching. In *Proceedings CVPR IEEE computer society conference on computer vision and pattern recognition*, 358–363. Ieee.

Du, X.; Wang, Y.; Zhan, K.; and Yu, X. 2026. Mobile-GS: Real-time Gaussian Splatting for Mobile Devices. arXiv:2603.11531.

Fang, G.; and Wang, B. 2024a. Mini-splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision*, 165–181. Springer.

Fang, G.; and Wang, B. 2024b. Mini-splatting2: Building 360 scenes within minutes via aggressive gaussian densification. *arXiv preprint arXiv:2411.12788*.

Fang, G.; and Wang, B. 2025. Efficient Scene Modeling Via Structure-Aware and Region-Prioritized 3D Gaussians. *IEEE TPAMI*.

Feigl, T.; Porada, A.; Steiner, S.; Löffler, C.; Mutschler, C.; and Philippsen, M. 2020. Localization Limitations of ARCore, ARKit, and Hololens in Dynamic Large-scale Industry Environments. In *VISIGRAPP (1: GRAPP)*, 307–318.

Fife, W. S.; and Archibald, J. K. 2012. Improved census transforms for resource-optimized stereo vision. *IEEE Transactions on Circuits and Systems for Video Technology*, 23(1): 60–73.

Fridovich-Keil, S.; Yu, A.; Tancik, M.; Chen, Q.; Recht, B.; and Kanazawa, A. 2022. Plenoxels: Radiance fields without neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 5501–5510.

Gao, K.; Gao, Y.; He, H.; Lu, D.; Xu, L.; and Li, J. 2022. NeRF: Neural Radiance Field in 3D Vision, A Comprehensive Review. *arXiv preprint arXiv:2210.00379*.

Guo, W.; Wang, B.; and Chen, L. 2025. NeuV-SLAM: Fast neural multiresolution voxel optimization for RGBD dense SLAM. *IEEE Transactions on Multimedia*.

Hanson, A.; Tu, A.; Lin, G.; Singla, V.; Zwicker, M.; and Goldstein, T. 2025. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, 21537–21546.

Hirschmuller, H. 2005. Accurate and efficient stereo processing by semi-global matching and mutual information. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, volume 2, 807–814. IEEE.

Höllein, L.; Božič, A.; Zollhöfer, M.; and Nießner, M. 2025. 3dgs-lm: Faster gaussian-splatting optimization with levenberg-marquardt. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 26740–26750.

Huang, B.; Yu, Z.; Chen, A.; Geiger, A.; and Gao, S. 2024. 2D Gaussian Splatting for Geometrically Accurate Radiance Fields. *ACM SIGGRAPH 2024 Conference Proceedings*.

Jung, J.; Han, J.; An, H.; Kang, J.; Park, S.; and Kim, S. 2024. Relaxing Accurate Initialization Constraint for 3D Gaussian Splatting. *arXiv preprint arXiv:2403.09413*.

Kerbl, B.; Kopanas, G.; Leimkühler, T.; and Drettakis, G. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Trans. Graph.*, 42(4).

Kim, P.; Kim, J.; Song, M.; Lee, Y.; Jung, M.; and Kim, H.-G. 2022. A benchmark comparison of four off-the-shelf proprietary visual–inertial odometry systems. *Sensors*, 22(24): 9873.

Kotovenko, D.; Grebenkova, O.; and Ommer, B. 2025. EDGS: Eliminating Densification for Efficient Convergence of 3DGS. *arXiv preprint arXiv:2504.13204*.

Lan, L.; Shao, T.; Lu, Z.; Zhang, Y.; Jiang, C.; and Yang, Y. 2025. 3dgs2: Near second-order converging 3d gaussian splatting. In *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, 1–10.

Lin, W.; Feng, Y.; and Zhu, Y. 2025. Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 669–682.

Liu, X.; Shi, D.; and Yang, S. 2025. MobileGaussian: Efficient 3D Gaussian-based Open Vocabulary Scene Understanding on Mobile Devices. In *Proceedings of the 2025 2nd International Conference on Computer Network and Cloud Computing*, 76–81.

Liu, Z.; Zhu, H.; Li, X.; Wang, Y.; Shi, Y.; Li, W.; Leng, J.; Guo, M.; and Feng, Y. 2025. Voyager: Real-Time Splatting City-Scale 3D Gaussians on Your Phone. *arXiv e-prints*, arXiv–2506.

Lu, T.; Dhiman, A.; Srinath, R.; Arslan, E.; Xing, A.; Xiangli, Y.; Babu, R. V.; and Sridhar, S. 2024. Turbo-GS: Accelerating 3D Gaussian Fitting for High-Quality Radiance Fields. *arXiv preprint arXiv:2412.13547*.

Mallick, S. S.; Goel, R.; Kerbl, B.; Steinberger, M.; Carrasco, F. V.; and De La Torre, F. 2024. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, 1–11.

Mildenhall, B.; Srinivasan, P. P.; Ortiz-Cayon, R.; Kalantari, N. K.; Ramamoorthi, R.; Ng, R.; and Kar, A. 2019. Local light field fusion: Practical view synthesis with prescriptive sampling guidelines. *ACM Transactions on Graphics (ToG)*, 38(4): 1–14.

Mildenhall, B.; Srinivasan, P. P.; Tancik, M.; Barron, J. T.; Ramamoorthi, R.; and Ng, R. 2021. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1): 99–106.

Müller, T.; Evans, A.; Schied, C.; and Keller, A. 2022. Instant Neural Graphics Primitives with a Multiresolution Hash Encoding. *ACM Trans. Graph.*, 41(4).

Mur-Artal, R.; and Tardós, J. D. 2017. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE transactions on robotics*, 33(5): 1255–1262.

Peng, Z.; Shao, T.; Liu, Y.; Zhou, J.; Yang, Y.; Wang, J.; and Zhou, K. 2024. Rtg-slam: Real-time 3d reconstruction at scale using gaussian splatting. In *ACM SIGGRAPH 2024 conference papers*, 1–11.

Picard, Q.; Chevobbe, S.; Darouich, M.; and Didier, J.-Y. 2023. A survey on real-time 3D scene reconstruction with SLAM methods in embedded systems. *arXiv preprint arXiv:2309.05349*.

Qin, T.; Li, P.; and Shen, S. 2018. VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator. *IEEE Transactions on Robotics*, 34(4): 1004–1020.

Ren, S.; Wen, T.; Fang, Y.; and Lu, B. 2025. FastGS: Training 3D Gaussian Splatting in 100 Seconds. *arXiv preprint arXiv:2511.04283*.

Rojas, S.; Zarzar, J.; Pérez, J. C.; Sanchez-Riera, J.; Rodríguez, A.; Segu, F.; and Moreno-Noguer, F. 2023. ReReND: Real-Time Rendering of NeRFs across Devices. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

Sanim, M. M. R.; Shu, Z.; Afsharmanesh, B.; Mirian, A.; Guan, J.; Niu, W.; Ren, B.; and Agrawal, G. 2025. Optimizing 3D Gaussian Splattering for Mobile GPUs. In *2025 34th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 359–371. IEEE.

Schönberger, J. L.; and Frahm, J.-M. 2016. Structure-from-Motion Revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Sun, C.; Sun, M.; and Chen, H.-T. 2022. Direct Voxel Grid Optimization: Super-Fast Convergence for Radiance Fields Reconstruction. In *CVPR*.

Tewari, A.; Thies, J.; Mildenhall, B.; Srinivasan, P.; Tretschk, E.; Yifan, W.; Lassner, C.; Sitzmann, V.; Martin-Brualla, R.; Lombardi, S.; et al. 2022. Advances in neural rendering. In *Computer Graphics Forum*, volume 41, 703–735. Wiley Online Library.

Triggs, B.; McLauchlan, P. F.; Hartley, R. I.; and Fitzgibbon, A. W. 1999. Bundle Adjustment — A Modern Synthesis. In *Vision Algorithms: Theory and Practice*. Springer.

Barron, J. T.; Mildenhall, B.; Verbin, D.; Srinivasan, P. P.; and Hedman, P. 2023. Zip-NeRF: Anti-Aliased Grid-Based Neural Radiance Fields. In *ICCV*.

Wenzhi, G.; Haiyang, B.; Yuanqu, M.; Jia, L.; and Lijun, C. 2023. FVLoc-NeRF: Fast Vision-Only Localization within Neural Radiation Field. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3329–3334. IEEE.

Yan, C.; Qu, D.; Xu, D.; Zhao, B.; Wang, Z.; Wang, D.; and Li, X. 2024. GS-SLAM: Dense Visual SLAM with 3D Gaussian Splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Yao, Y.; Luo, Z.; Li, S.; Fang, T.; and Quan, L. 2018. MVSNet: Depth Inference for Unstructured Multi-view Stereo. In *European Conference on Computer Vision (ECCV)*.